

ÉCOLE D'INGÉNIEURS SUP GALILÉE

Rapport du TP5 de Base De Données Avancées

Encadrant :
Mr YUCEF

Élève :
Maryse GOEH-AKUE
N° étudiant : 12101028

Exercice 1 - Atomicité d'une transaction

Dans cet exercice, l'objectif est d'étudier la propriété d'**atomicité** des transactions en base de données. Pour cela, il faut commencer chaque session SQL*Plus par la commande suivante :

```
SET AUTOCOMMIT OFF
```

Cette commande est importante car elle empêche Oracle de valider automatiquement les modifications après chaque instruction. Ainsi, les changements effectués restent temporaires tant qu'une commande **COMMIT** n'a pas été exécutée. On peut alors observer clairement l'effet d'un **ROLLBACK**, d'une fermeture de session ou d'une modification de structure.

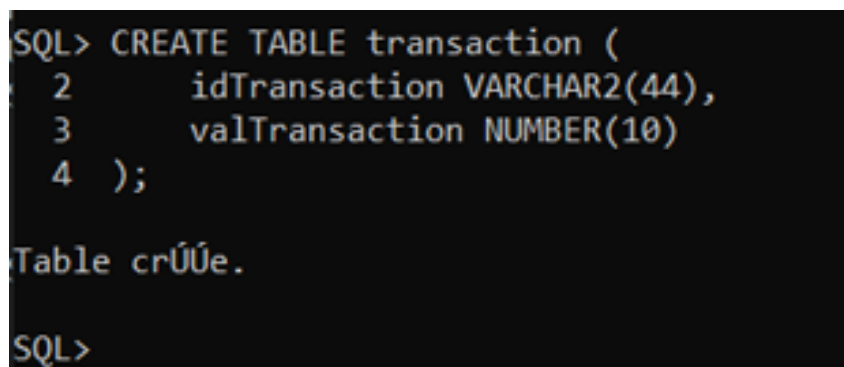
Question 1. Création de la table transaction

La première étape consiste à créer la table **transaction**, qui sera utilisée pour toutes les manipulations de l'exercice.

```
CREATE TABLE transaction (  
    idTransaction VARCHAR2(44),  
    valTransaction NUMBER(10)  
);
```

Cette table contient :

- **idTransaction** : un identifiant de transaction ;
- **valTransaction** : une valeur numérique associée.



```
SQL> CREATE TABLE transaction (  
2     idTransaction VARCHAR2(44),  
3     valTransaction NUMBER(10)  
4 );  
  
Table créée.  
  
SQL>
```

FIGURE 1 – Création de la table **transaction**

La capture montre que la table a bien été créée dans la session SQL*Plus. Cette étape prépare simplement l'environnement de travail pour les questions suivantes.

Question 2. Insertions, modification, suppression puis ROLLBACK

Dans cette question, on effectue plusieurs opérations successives sur la table :

- insertion de lignes ;
- modification d'une ligne existante ;

- suppression d'une autre ligne ;
- annulation complète de ces changements avec ROLLBACK.

Exemple d'instructions exécutées :

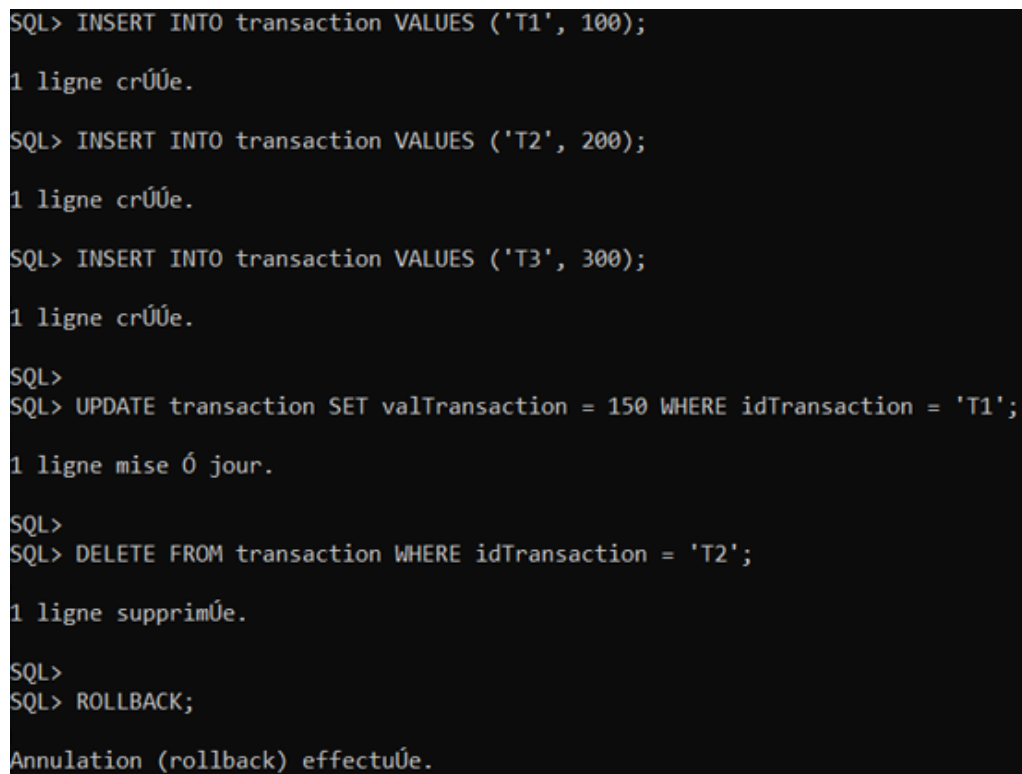
```
INSERT INTO transaction VALUES ('t1', 100);
INSERT INTO transaction VALUES ('t2', 200);
INSERT INTO transaction VALUES ('t3', 300);
```

```
UPDATE transaction
SET valTransaction = 150
WHERE idTransaction = 't1';
```

```
DELETE FROM transaction
WHERE idTransaction = 't2';
```

```
ROLLBACK;
```

```
SELECT * FROM transaction;
```



```
SQL> INSERT INTO transaction VALUES ('T1', 100);
1 ligne créée.

SQL> INSERT INTO transaction VALUES ('T2', 200);
1 ligne créée.

SQL> INSERT INTO transaction VALUES ('T3', 300);
1 ligne créée.

SQL>
SQL> UPDATE transaction SET valTransaction = 150 WHERE idTransaction = 'T1';
1 ligne mise à jour.

SQL>
SQL> DELETE FROM transaction WHERE idTransaction = 'T2';
1 ligne supprimée.

SQL>
SQL> ROLLBACK;

Annulation (rollback) effectuée.
```

FIGURE 2 – Insertions, modification, suppression puis annulation avec ROLLBACK

Sur cette première capture, on observe que :

- plusieurs lignes sont insérées ;
- la valeur de la transaction `t1` est modifiée ;
- la ligne `t2` est supprimée ;
- enfin, la commande `ROLLBACK` est exécutée.

Après cela, on vérifie le contenu de la table :

```
SQL> SELECT * FROM transaction;

aucune ligne sélectionnée

SQL>
```

FIGURE 3 – Résultat du SELECT après ROLLBACK

Le résultat indique qu'aucune ligne n'est présente dans la table. Cela signifie que toutes les modifications effectuées depuis le début de la transaction ont été annulées.

Conclusion : La commande ROLLBACK annule toutes les opérations non validées de la transaction en cours. Dans ce cas, les insertions, la modification et la suppression disparaissent totalement. Cela montre bien la propriété d'**atomicité** : soit l'ensemble des opérations est validé, soit aucune ne l'est.

Question 3. Insertions puis fermeture avec quit

Dans cette partie, on insère à nouveau des lignes dans la table, mais on ferme ensuite SQL*Plus avec la commande quit sans exécuter de COMMIT.

```
INSERT INTO transaction VALUES ('T100', 1000);
INSERT INTO transaction VALUES ('T200', 2000);
quit;
```

```
SQL> INSERT INTO transaction VALUES ('T100', 1000);

1 ligne créée.

SQL> INSERT INTO transaction VALUES ('T200', 2000);

1 ligne créée.

SQL> quit
Déconnecté de Oracle Database 10g Express Edition Release 10.2.0.1.0 - Production
```

FIGURE 4 – Insertions suivies de la fermeture de SQL*Plus avec quit

La capture montre que les deux insertions ont bien été prises en compte dans la session courante. Cependant, comme aucun COMMIT n'a été effectué avant de quitter, ces changements ne sont pas encore enregistrés définitivement dans la base.

Après reconnexion, on consulte le contenu de la table :

```
SQL> SELECT * FROM transaction;

aucune ligne sélectionnée
```

FIGURE 5 – Vérification après fermeture sans COMMIT

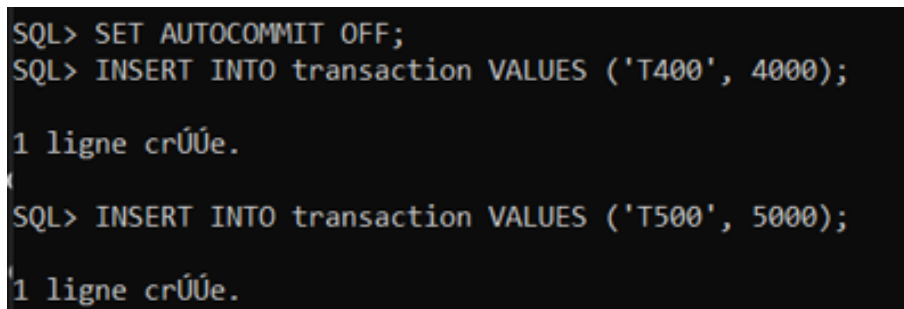
On constate que la table est vide. Les lignes insérées avant la fermeture n'ont donc pas été conservées.

Conclusion : En mode `AUTO COMMIT OFF`, quitter la session sans validation revient à perdre la transaction en cours. Les opérations non validées ne deviennent jamais permanentes dans la base de données.

Question 4. Insertion puis fermeture brutale de SQL*Plus

Ici, on étudie un cas similaire au précédent, mais avec une fermeture brutale de la session au lieu d'une fermeture normale avec `quit`. On commence par insérer quelques lignes :

```
INSERT INTO transaction VALUES ('T400', 4000);  
INSERT INTO transaction VALUES ('T500', 5000);
```

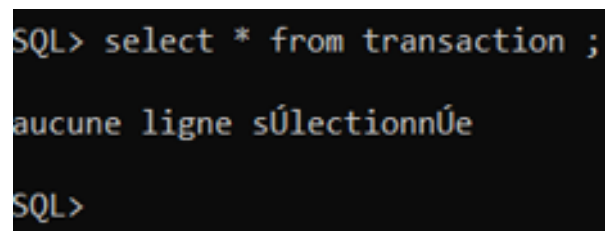


```
SQL> SET AUTOCOMMIT OFF;  
SQL> INSERT INTO transaction VALUES ('T400', 4000);  
  
1 ligne créée.  
  
SQL> INSERT INTO transaction VALUES ('T500', 5000);  
  
1 ligne créée.
```

FIGURE 6 – Insertions effectuées avant la fermeture brutale

Les lignes sont bien ajoutées dans la session en cours. Toutefois, comme elles ne sont toujours pas validées par un `COMMIT`, elles restent provisoires.

Après reconnexion, on effectue un `SELECT` sur la table :



```
SQL> select * from transaction ;  
  
aucune ligne sélectionnée  
  
SQL>
```

FIGURE 7 – Contenu de la table après reconnexion

Là encore, aucune ligne n'apparaît dans la table.

Cela signifie qu'Oracle a annulé automatiquement les changements qui n'avaient pas été validés avant l'interruption de la session.

Conclusion : Une fermeture brutale de SQL*Plus avant `COMMIT` entraîne la perte de la transaction en cours. Les données saisies ne sont donc pas préservées. Cela confirme qu'une transaction non validée ne peut pas survivre à la fin de la session.

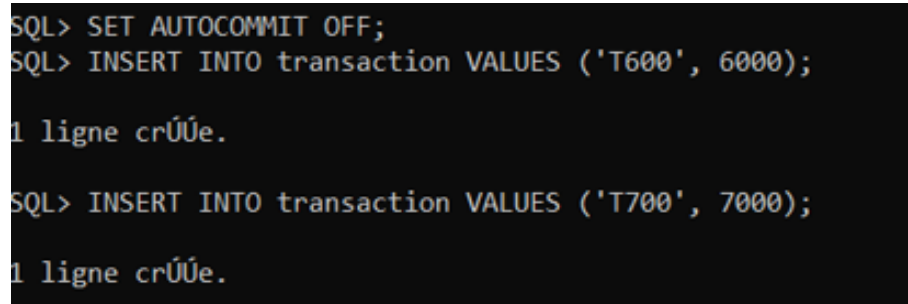
Question 5. Modification de structure puis ROLLBACK

Dans cette question, on veut comparer deux types d'opérations :

- les opérations de manipulation de données (INSERT, UPDATE, DELETE);
- les opérations de définition de structure (ALTER TABLE, CREATE, DROP).

On commence par insérer quelques lignes :

```
INSERT INTO transaction VALUES ('T600', 6000);  
INSERT INTO transaction VALUES ('T700', 7000);
```

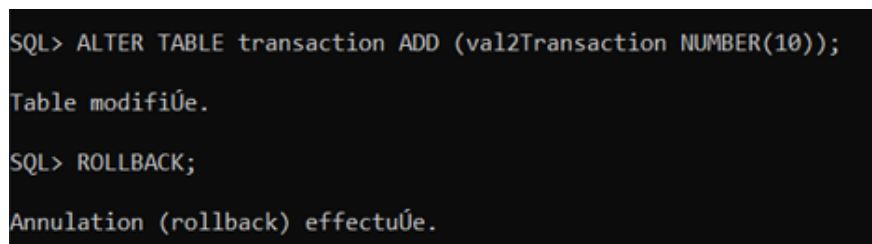


```
SQL> SET AUTOCOMMIT OFF;  
SQL> INSERT INTO transaction VALUES ('T600', 6000);  
  
1 ligne créée.  
  
SQL> INSERT INTO transaction VALUES ('T700', 7000);  
  
1 ligne créée.
```

FIGURE 8 – Insertions effectuées avant modification de la structure

Ensuite, on modifie la structure de la table en ajoutant une nouvelle colonne :

```
ALTER TABLE transaction  
ADD val2Transaction NUMBER(10);  
  
ROLLBACK;
```



```
SQL> ALTER TABLE transaction ADD (val2Transaction NUMBER(10));  
  
Table modifiée.  
  
SQL> ROLLBACK;  
  
Annulation (rollback) effectuée.
```

FIGURE 9 – Commande ALTER TABLE suivie d'un ROLLBACK

Après cela, on vérifie la structure de la table avec la commande DESC transaction :

```

SQL> SELECT * FROM transaction;

aucune ligne sélectionnée

SQL> DESC transaction;
  Nom                                NULL ?    Type
-----
IDTRANSACTION                       VARCHAR2(44)
VALTRANSACTION                       NUMBER(10)
VAL2TRANSACTION                       NUMBER(10)

```

FIGURE 10 – Description de la table après ROLLBACK

On remarque que :

- la table ne contient toujours aucune ligne ;
- la nouvelle colonne **VAL2TRANSACTION** est bien présente.

Cela montre que le **ROLLBACK** a annulé les insertions, mais n'a pas annulé la modification de structure.

Conclusion : Les commandes DDL comme **CREATE**, **ALTER** ou **DROP** provoquent un **COMMIT** implicite dans Oracle. Par conséquent, leur effet devient immédiatement permanent et ne peut pas être annulé par un **ROLLBACK**. En revanche, les insertions non validées, elles, peuvent encore être annulées.

Question 6. Conclusion générale

Cet exercice met en évidence plusieurs points essentiels sur les transactions en Oracle.

D'abord, une **session** correspond à une connexion ouverte entre un utilisateur et le système de gestion de base de données. Pendant cette session, l'utilisateur peut exécuter plusieurs commandes SQL.

Ensuite, une **transaction** est un ensemble d'opérations exécutées comme une seule unité logique. Tant qu'elle n'est pas validée, ses effets restent temporaires.

Pour rendre les modifications définitives, on utilise la commande :

COMMIT

Pour annuler toutes les modifications non encore validées, on utilise :

ROLLBACK

Enfin, cet exercice montre bien la propriété d'**atomicité** :

- soit toutes les modifications de la transaction sont conservées après validation ;
- soit aucune ne l'est si la transaction est annulée ou interrompue avant **COMMIT**.

L'atomicité garantit donc qu'une transaction ne laisse jamais la base dans un état partiellement modifié.

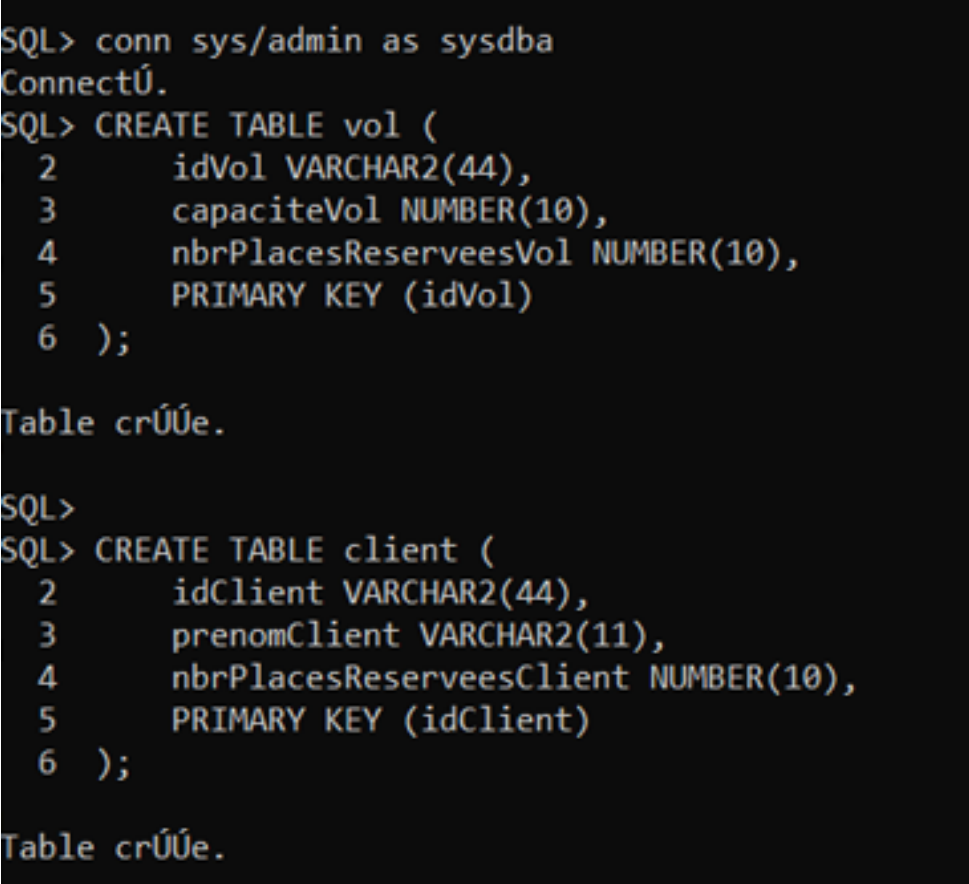
Exercice 2 - Transactions concurrentes

Dans cet exercice, on étudie le comportement de deux transactions exécutées en parallèle, afin de comprendre les notions d'**isolation**, de **cohérence** et les problèmes liés à la concurrence.

Question 1. Création du schéma

On commence par créer les deux tables nécessaires : une table `vol` et une table `client`.

```
CREATE TABLE vol (  
    idVol VARCHAR2(44),  
    capaciteVol NUMBER(10),  
    nbrPlacesReserveesVol NUMBER(10),  
    PRIMARY KEY (idVol)  
);  
  
CREATE TABLE client (  
    idClient VARCHAR2(44),  
    prenomClient VARCHAR2(11),  
    nbrPlacesReserveesClient NUMBER(10),  
    PRIMARY KEY (idClient)  
);
```



```
SQL> conn sys/admin as sysdba  
Connecté.  
SQL> CREATE TABLE vol (  
2     idVol VARCHAR2(44),  
3     capaciteVol NUMBER(10),  
4     nbrPlacesReserveesVol NUMBER(10),  
5     PRIMARY KEY (idVol)  
6 );  
  
Table créée.  
  
SQL>  
SQL> CREATE TABLE client (  
2     idClient VARCHAR2(44),  
3     prenomClient VARCHAR2(11),  
4     nbrPlacesReserveesClient NUMBER(10),  
5     PRIMARY KEY (idClient)  
6 );  
  
Table créée.
```

FIGURE 11 – Création des tables `vol` et `client`

Ensuite, on insère des données initiales :

```
INSERT INTO vol VALUES ('V001', 100, 0);
```



```
INSERT INTO client VALUES ('C001', 'John', 2);
INSERT INTO client VALUES ('C002', 'Alice', 3);
```

```
SQL> INSERT INTO vol (idVol, capaciteVol, nbrPlacesReserveesVol) VALUES ('V001', 100, 0);
1 ligne cr  e.
SQL> INSERT INTO client (idClient, prenomClient, nbrPlacesReserveesClient) VALUES ('C001', 'John', 2);
1 ligne cr  e.
SQL> INSERT INTO client (idClient, prenomClient, nbrPlacesReserveesClient) VALUES ('C002', 'Alice', 3);
1 ligne cr  e.
SQL>
```

FIGURE 12 – Insertion des donn  es initiales

Question 2. Isolation des transactions

On ex  cute deux transactions en parall  le :

- T1 : effectue une mise    jour sans faire COMMIT
- T2 : tente de lire les donn  es

```
UPDATE client SET nbrPlacesReserveesClient = 5
WHERE idClient = 'C001';
```

```
SELECT * FROM client;
```

```
SQL> UPDATE client SET nbrPlacesReserveesClient = 5 WHERE idClient = 'C001';
0 ligne(s) mise(s)    jour.
SQL> SELECT * FROM client;
aucune ligne s  lectionn  e
```

Dans la deuxi  me session :

```
SQL> commit ;
Validation effectu  e.
```

On observe que :

- la transaction T1 voit ses modifications
- la transaction T2 ne voit rien

Explication : Oracle utilise par d  faut le niveau d'isolation READ COMMITTED. Cela signifie que seules les donn  es valid  es (COMMIT) sont visibles par les autres transactions.

Conclusion : Une transaction voit ses propres modifications, mais les autres transactions ne voient que les donn  es valid  es.

Question 3. Effet du ROLLBACK

On effectue une modification dans T1 puis on annule avec ROLLBACK.

```
SQL> select * from client ;
```

IDCLIENT	NBRPLACESRESERVEESCLIENT	PRENOMCLIEN
C001	2	John
C002	3	Alice

Puis on vérifie les données :

```
SQL> select * from client ;
```

IDCLIENT	NBRPLACESRESERVEESCLIENT	PRENOMCLIEN
C001	5	John
C002	3	Alice

Explication : Le ROLLBACK annule toutes les modifications de la transaction en cours.

Conclusion : La base revient à son état initial, comme si la transaction n'avait jamais existé.

Question 4. Effet du COMMIT

On refait la modification mais cette fois avec validation :

```
UPDATE client SET nbrPlacesReserveesClient = 5
WHERE idClient = 'C001';
```

```
COMMIT;
```

```
SQL> UPDATE client SET nbrPlacesReserveesClient = 5 WHERE idClient = 'C001';
1 ligne mise à jour.
SQL>
```

Puis :

```
SQL> SELECT * FROM client WHERE idClient = 'C001';
```

aucune ligne sélectionnée

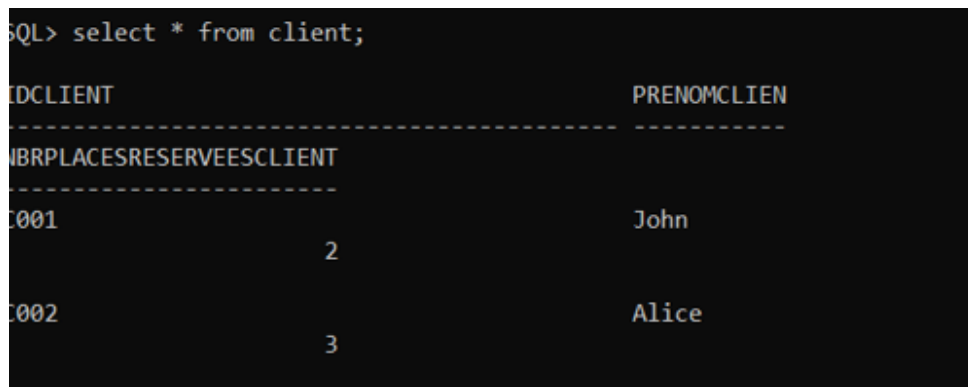
Explication : Le COMMIT rend les modifications permanentes.

Conclusion : Une fois validées, les données ne peuvent plus être annulées. Cela correspond à la propriété de **durabilité**.

Question 5. Mise à jour perdue (READ COMMITTED)

On exécute deux transactions en parallèle :

- T1 réserve 2 places
- T2 réserve 3 places



```
SQL> select * from client;
```

IDCLIENT	PRENOMCLIENT
001	John
002	Alice

Résultat observé :

- les clients totalisent 5 places
- le vol n'enregistre que 3 places

Explication : Chaque transaction lit les anciennes valeurs avant que l'autre ne valide. Cela provoque une incohérence appelée **mise à jour perdue**.

Conclusion : Le niveau READ COMMITTED ne garantit pas une cohérence parfaite.

Question 6. Mode SERIALIZABLE

On change le niveau d'isolation :

```
ALTER SESSION SET ISOLATION_LEVEL = SERIALIZABLE;
```

Puis on refait le test.

```
SQL> UPDATE client SET nbrPlacesReserveesClient = 5 WHERE idClient = 'C001';
1 ligne mise à jour.
SQL> commit ;
Validation effectuée.
SQL> rollback
2 ;
Annulation (rollback) effectuée.
SQL> select * from client;
```

IDCLIENT		PRENOMCLIEN
C001	5	John
C002	3	Alice

Explication : En mode **SERIALIZABLE**, Oracle empêche les incohérences.

Conclusion :

- une transaction peut être bloquée ou rejetée
- mais la base reste cohérente

Conclusion générale

Cet exercice montre que :

- le mode **READ COMMITTED** est rapide mais peut provoquer des anomalies
- le mode **SERIALIZABLE** garantit la cohérence

Oracle utilise un mécanisme appelé **MVCC** (Multi-Version Concurrency Control), qui permet de gérer les transactions concurrentes sans blocage systématique.